

Data Exploration with **pandas**

Eunji Kim

eunjikim@cau.ac.kr

Outline

- Load and Save Data
- Access and Manipulate Data
- Summary Statistics

Load Data

Iris dataset



- 150 instances
- 3 categories (setosa, versicolor, virginica)
- 4 numerical attributes



Can be obtained from the UCI Machine Learning Repository:

<http://www.ics.uci.edu/~mlearn/MLRepository.html>

Data File

If you open `Iris.csv` with a text editor, you can see comma-separated values:

```
Id,SepalLengthCm,SepalWidthCm,PetalLengthCm,PetalWidthCm,Species
1,5.1,3.5,1.4,0.2,Iris-setosa
2,4.9,3.0,1.4,0.2,Iris-setosa
3,4.7,3.2,1.3,0.2,Iris-setosa
4,4.6,3.1,1.5,0.2,Iris-setosa
5,5.0,3.6,1.4,0.2,Iris-setosa
6,5.4,3.9,1.7,0.4,Iris-setosa
7,4.6,3.4,1.4,0.3,Iris-setosa
8,5.0,3.4,1.5,0.2,Iris-setosa
9,4.4,2.9,1.4,0.2,Iris-setosa
10,4.9,3.1,1.5,0.1,Iris-setosa
```

CSV (Comma Separated Values)

Records of data represented with comma separated values

- Each line is a new record
- Each record consists of one or more columns, separated by commas

How to load data?



Pandas official website: <https://pandas.pydata.org/>

Import `pandas` and use it!

```
import pandas as pd  
  
df = pd.read_csv('./Iris.csv')
```

Here, variable `df` refers to a `pd.DataFrame` object

```
print(type(df)) # pandas.core.frame.DataFrame
```

Check your dataframe using the variable `df`

`df`

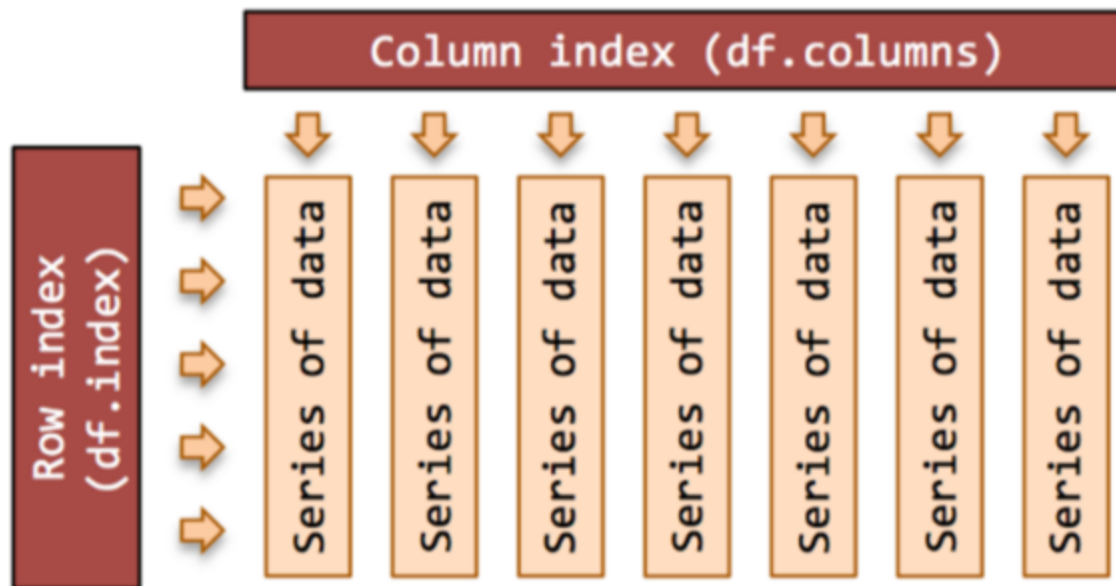
	<code>Id</code>	<code>SepalLengthCm</code>	<code>SepalWidthCm</code>	<code>PetalLengthCm</code>	<code>PetalWidthCm</code>	<code>Species</code>
<code>0</code>	1	5.1	3.5	1.4	0.2	Iris-setosa
<code>1</code>	2	4.9	3.0	1.4	0.2	Iris-setosa
<code>2</code>	3	4.7	3.2	1.3	0.2	Iris-setosa
<code>3</code>	4	4.6	3.1	1.5	0.2	Iris-setosa
<code>4</code>	5	5.0	3.6	1.4	0.2	Iris-setosa
<code>...</code>	<code>...</code>	<code>...</code>	<code>...</code>	<code>...</code>	<code>...</code>	<code>...</code>
<code>145</code>	146	6.7	3.0	5.2	2.3	Iris-virginica
<code>146</code>	147	6.3	2.5	5.0	1.9	Iris-virginica
<code>147</code>	148	6.5	3.0	5.2	2.0	Iris-virginica
<code>148</code>	149	6.2	3.4	5.4	2.3	Iris-virginica
<code>149</code>	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

`pd.DataFrame` class

A tabular, spreadsheet-like data structure

- a collection of columns, where each column is represented as a `pd.Series`



Columns Names and Row Indices

- Check column names

```
# check column names
>>> df.columns

Index(['Id', 'SepalLengthCm', 'SepalWidthCm',
       'PetalLengthCm', 'PetalWidthCm', 'Species'], dtype='object')
```

- Check row indices

```
# check row indices
>>> df.index

RangeIndex(start=0, stop=150, step=1)
```

Access Data

Access to a Column

How to access a column in a dataframe?

```
# access a column (=pd.Series)  
df['SepalLengthCm']
```

Output:

```
0 5.1  
1 4.9  
2 4.7  
3 4.6  
4 5.0  
...  
145 6.7  
146 6.3  
147 6.5  
148 6.2  
149 5.9  
Name: SepalLengthCm, Length: 150, dtype: float64
```

Access to a Row

Use `.loc[]` (label-based) or `.iloc[]` (integer position-based).

Example:

```
df.loc[2]      # Access row with index label 2  
df.iloc[2]     # Access the 3rd row (Python-style index)
```

Access to a Specific Value

Use `.loc[]` (label-based) or `.iloc[]` (integer position-based).

Example:

```
df.loc[2, 'Species']    # Species value in row with index label 2  
df.iloc[2, 4]           # value in 3rd row, 5th column
```

Summary Statistics

Check Unique Values & Counts (Categorical Column)

For a categorical column, we can get its unique values and their counts

- `.unique()` : See all unique categories

```
# See unique species names
>>> df['Species'].unique()

array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)
```

- `.value_counts()` : Count how many times each unique value appears

```
# Count instances of each species
>>> df['Species'].value_counts()

Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: Species, dtype: int64
```

Mean, std, median (Numerical Columns)

How to check the summary statistics of a column in a pandas DataFrame?

- E.g., `SepalLengthCm` column

```
df['SepalLengthCm'].mean()    # mean
df['SepalLengthCm'].std()     # stdev
df['SepalLengthCm'].median()  # median
```



Note: You can also use the `numpy` package to get the same results:

```
import numpy as np

np.mean(df['SepalLengthCm'])    # mean
np.std(df['SepalLengthCm'])     # stdev
np.median(df['SepalLengthCm'])  # median
```

(advanced) Check the mean of four numeric columns

Use `for` loop

```
# (advanced) check the mean of four numeric columns
for col in df.columns[:-1]:
    colmean = df[col].mean()
    print(f"The mean of {col} is {colmean:.2f}")
```



Here, we can use `:.2f` to print a number rounded to 2 decimal places.

Grouping Data

How to get summary statistics per category? Use `.groupby()`

```
# Group by the 'Species' column
>>> df.groupby('Species')

<pandas.core.groupby.generic.DataFrameGroupBy object ...>
```

You can then calculate statistics for each group. Here is an example of calculating the mean of all numeric columns for each species

```
df.groupby('Species').mean()
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
Species				
Iris-setosa	5.006	3.418	1.464	0.244
Iris-versicolor	5.936	2.770	4.260	1.326
Iris-virginica	6.588	2.974	5.552	2.026

This will show the mean of all numeric columns for each species.

DataFrame.describe()

A powerful method to see summary statistics

```
# Summary Statistics  
df.describe(include='all')
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
count	150.000000	150.000000	150.000000	150.000000	150.000000	150
unique	NaN	NaN	NaN	NaN	NaN	3
top	NaN	NaN	NaN	NaN	NaN	Iris-setosa
freq	NaN	NaN	NaN	NaN	NaN	50
mean	75.500000	5.843333	3.054000	3.758667	1.198667	NaN
std	43.445368	0.828066	0.433594	1.764420	0.763161	NaN
min	1.000000	4.300000	2.000000	1.000000	0.100000	NaN
25%	38.250000	5.100000	2.800000	1.600000	0.300000	NaN
50%	75.500000	5.800000	3.000000	4.350000	1.300000	NaN
75%	112.750000	6.400000	3.300000	5.100000	1.800000	NaN
max	150.000000	7.900000	4.400000	6.900000	2.500000	NaN

**Summary
Statistics**

Filter Rows

Filter Rows based on column values

This is also called Boolean Indexing.

1. Create a condition (a boolean Series).
2. Pass that condition back to the DataFrame.

```
# 1. Get all rows where PetalLengthCm > 6.0
df[df['PetalLengthCm'] > 6.0]

# 2. Get all rows where Species is 'Iris-setosa'
df[df['Species'] == 'Iris-setosa']
```

 Rule of thumb: If your function only needs one value, use `Series.apply()`. If it needs multiple values from the same row, use `DataFrame.apply(axis=1)`.

(advanced) Multiple Conditions

Use `&` for AND and `|` for OR.

⚠ Important: You must use parentheses `()` around each condition.

```
# Species is 'Iris-virginica' AND PetalWidthCm > 2.2
df[
    (df['Species'] == 'Iris-virginica') &
    (df['PetalWidthCm'] > 2.2)
]
```

Add/Remove Columns

Add a New column

You can create a new column by simple assignment using `[]`.

Example 1: Based on existing columns

```
# Create a 'PetalArea' column
df['PetalArea'] = df['PetalLengthCm'] * df['PetalWidthCm']
# Create an 'Id' column
df['Id'] = df.index
```

Example 2: With a constant value

```
# Add a column with a single value (it will be broadcast to all rows)
df['Source'] = 'UCI Repository'
```

Remove Columns with `.drop()`

The most flexible way to remove columns is the `.drop()` method.

- You must specify `axis=1` to tell pandas you are dropping a column.
- By default, it returns a new DataFrame.

```
# Drop the 'Id' column
# This does NOT change the original df
df_no_id = df.drop('Id', axis=1)

# To permanently remove it, you must re-assign the variable:
df = df.drop('Id', axis=1)
```

To drop multiple columns, pass a list:

```
# Let's drop the 'PetalArea' and 'Source' columns
df = df.drop(['PetalArea', 'Source'], axis=1)
```

Remove a Column (Alternative: `del`)

You can also use Python's `del` keyword for a simple, in-place deletion.

```
# Let's re-add 'PetalArea' just to delete it
df['PetalArea'] = df['PetalLengthCm'] * df['PetalWidthCm']
print("Columns before:", df.columns)

# Now delete it in-place
del df['PetalArea']

# The df is changed immediately
print("Columns after:", df.columns)
```

Apply Functions using `.apply()`

apply()

A powerful method to **apply a custom function to your data**.

- Use it when simple operations (`+` , `*` , `.mean()`) aren't enough.
- It's one of the most flexible (and useful) tools in `pandas` .

There are two main types you will use:

1. `Series.apply()` : Applies a function to each element in a single column.
2. `DataFrame.apply()` : Applies a function along an axis (either to each column or each row).

Series.apply() (On a Single Column)

Applies a function to every element in a Series (a column).

Output is a new `Series` of the same size.

Example: Shorten Species names

```
# We want 'Iris-setosa' -> 'setosa'
# The function takes one value (x) and returns one value
def shorten_species(x):
    return x.split('-')[1] # 'Iris-setosa' splits into ['Iris', 'setosa']

# Apply this function to the 'Species' column
df['ShortSpecies'] = df['Species'].apply(shorten_species)
```

```
>>> df[['Species', 'ShortSpecies']].head()
   Species ShortSpecies
0  Iris-setosa      setosa
1  Iris-setosa      setosa
2  Iris-setosa      setosa
3  Iris-setosa      setosa
4  Iris-setosa      setosa
```

Series.apply() with lambda

For simple, one-line functions, using a `lambda` (anonymous) function is very common.

- Syntax: `lambda arguments: expression`

Example: Categorize Petal Length

```
# We want to label petals as 'short' or 'long'
# A lambda function is perfect for this
df['PetalSize'] = df['PetalLengthCm'].apply(
    lambda length: 'short' if length < 3.0 else 'long')
```

```
>>> df[['PetalLengthCm', 'PetalSize']].head()
   PetalLengthCm PetalSize
0             1.4    short
1             1.4    short
2             1.3    short
3             1.5    short
4             1.4    short
```

`DataFrame.apply()` (Row-wise / Column-wise)

Applies a function along an axis (rows or columns) of a DataFrame.

- `axis=0` (default): Apply function to each column.
 - Input to function is a `Series` (the whole column).
 - Example: `df.apply(np.mean, axis=0)` finds the mean of each column.
- `axis=1` : Apply function to each row.
 - Input to function is a `Series` (the whole row).
 - This is extremely useful for creating new columns based on other columns.

 You will use `df.apply(my_function, axis=1)` very often 😊.

DataFrame.apply(axis=1) Example

Let's create a new column, `PetalRatio`, which is the ratio of `PetalLengthCm` to `PetalWidthCm`.

```
# 1. Define a function that takes a row (x) as input
def calculate_petal_ratio(x):
    ratio = x['PetalLengthCm'] / x['PetalWidthCm']
    return ratio

# 2. Apply the function to every row (axis=1)
df['PetalRatio'] = df.apply(calculate_petal_ratio, axis=1)
```

```
>>> df[['PetalLengthCm', 'PetalWidthCm', 'PetalRatio']].head()
   PetalLengthCm  PetalWidthCm  PetalRatio
0             1.4            0.2    7.000000
1             1.4            0.2    7.000000
2             1.3            0.2    6.500000
3             1.5            0.2    7.500000
4             1.4            0.2    7.000000
```

(advanced) `apply(axis=1)` with `lambda`

You can achieve the same result in one line using a `lambda` function. Here, `row` is the variable name we chose for each row.

```
# The lambda function takes 'row' as input
# and returns the result of the calculation
df['PetalRatio'] = df.apply(
    lambda row: row['PetalLengthCm'] / row['PetalWidthCm'],
    axis=1)
```

```
>>> df['PetalRatio'].head()
0    7.0
1    7.0
2    6.5
3    7.5
4    7.0
Name: PetalRatio, dtype: float64
```

Save Data as a CSV File

Save a DataFrame as a CSV file

We can save a `pd.DataFrame` to a CSV file using `.to_csv()` method:

```
df.to_csv('./my_csv_filename.csv', index=False)
```



By setting `index=False`, we can exclude the index column from being created.

Thank You.